Implementation Overview

Real-time IMU gesture recognition on a Raspberry Pi + Sense HAT. A TensorFlow model is trained on a host, exported to TensorFlow Lite, and the C++ app below runs inference on the Pi - classifying gestures **A**, **B**, **C**, plus a **garbage/"no gesture"** class, and drawing the result on the 8×8 LED matrix.

This document describes the C++ application **as it currently stands** in src/.

Model contract

Defined in tflite_gesture_classifier.h:

Property	Value
Input shape	`[1, 50, 9]` - 50 timesteps × 9 features
Features	`accel_x/y/z`, `gyro_x/y/z`, `mag_x/y/z`
Output shape	`[1, 4]` - class scores
Classes	`0=A`, `1=B`, `2=C`, `3=?` (garbage)

Normalization is **baked into the model** (`tf.keras.layers.Normalization`), so the app feeds raw resampled sensor values directly - no scaling in C++.

Components

`main.cpp` - driver, CLI, and stream pipeline

main.cpp

Parses CLI args (ParseArgs) into one of three modes, loads the model once, and dispatches.

```
gesture_sensehat_demo --gesture   --model model.tflite --csv
test_gesture.csv
gesture_sensehat_demo --benchmark --model model.tflite --csv
test_gesture.csv --runs 100 --warmup 20
gesture_sensehat_demo --stream    --model model.tflite
gesture_sensehat_demo --no-sensehat ...   # suppress LED output
```

A bare positional argument is treated as the model path for backwards compatibility.

Mode 1 - `--gesture` (RunGestureInference)

Loads one pre-resampled gesture from CSV, runs a single inference, prints the predicted label, confidence, and the full per-class score vector, and (unless `--no-sensehat`) shows the symbol on the LED matrix.

Mode 2 - `--benchmark` (RunGestureBenchmark)

Runs `--warmup` untimed inferences, then times `--runs` inferences on the same CSV sample and reports total / average latency in ms. Output lines match the patterns expected by scripts/parse_perf_results.py.

Mode 3 - `--stream` (RunStreamMode)

The live pipeline. Loop body:

1. **Poll IMU** every `kPollIntervalMs` (60 ms) via `SenseHatImu::Read`.
2. **Maintain a ring buffer** of the most recent `kWindowRawSamples` (80) raw samples (`std::deque`, oldest popped from front).
3. **Trigger inference** when the buffer is full *and* `kWindowStride` (40) new samples have arrived since the last inference (≈ every 2.4 s of new data).
4. **Trim leading idle samples** before resampling: scan the window for the first sample whose dynamic acceleration `|‖accel‖ − 1g|` exceeds `kMotionThresholdG` (0.15 g), and resample only from that motion onset to the end. If fewer than 2 active samples remain, skip this window.
5. **Resample** the active region to exactly `50 × 9` via ResampleWindow.
6. **Classify** and apply the consensus / garbage state machine below.

Consensus & garbage state machine

State: `prev_class`, `consensus_count`, `garbage_count`.

- **Garbage window** (`cls == kGarbageClass`, i.e. 3): increment `garbage_count`; once it reaches **2**, reset consensus and clear the display. (A single garbage frame no longer wipes an in-progress gesture - it tolerates one spurious garbage classification.)
- **Real gesture window**: reset `garbage_count` to 0, then
  - if `cls == prev_class`, increment `consensus_count`; once it reaches `kConsensusRequired` (3) consecutive agreeing windows, **display the gesture** and **flush the entire buffer + reset all counters** so the next gesture starts from clean data;
  - otherwise start a new streak (`prev_class = cls`, `consensus_count = 1`).

`SIGINT`/`SIGTERM` set an atomic flag for clean shutdown; the display is cleared on exit.

Helpers

- LoadGestureInput - reads a pre-resampled CSV; takes the first 9 columns of each row as features (`kSkip = 0`), discarding the header.

- ResampleWindow - linear interpolation of N raw samples to `kGestureTimesteps × kGestureFeatures`, mirroring the Python training resampler so live and training data share the same temporal density.

`TfliteGestureClassifier` - TFLite wrapper

tflite_gesture_classifier.cpp (pImpl)

- **Load**: `FlatBufferModel::BuildFromFile` → `BuiltinOpResolver` → `InterpreterBuilder`; pins **1 thread** for deterministic latency; allocates tensors and validates exactly one input/output, supported types (`float32` or `int8`), and the expected `[1,50,9]` / `[1,4]` shapes.
- **CopyInput**: validates `input.size() == 50*9`. For `float32`, copies directly; for `int8`, **quantizes** with the tensor's `scale`/`zero_point` (clamped to int8 range).
- **ReadOutput**: returns the score vector directly for `float32`, or **dequantizes** for `int8`.
- **Predict**: copies input → `Invoke()` → reads scores → returns the argmax class index and its score as `confidence`. On any failure it sets `ok_ = false` and an error message.

`SenseHatImu` - IMU reader

sense_hat_imu.cpp (pImpl over RTIMULib)

- Constructs from `/etc/RTIMULib.ini` (installed by the `sense-hat` package) so axis orientation and calibration match the Python recording setup.
- `Read()` calls `IMURead()`, retrying up to 5× with 5 ms sleeps because the sensor ODR (~119 Hz) can lag a poll. Fills an ImuSample with a steady-clock timestamp and accel (g) / gyro (rad/s) / mag (µT) values.

`SenseHatDisplay` - LED matrix output

sense_hat_display.cpp

- Finds the framebuffer by scanning `/sys/class/graphics/fb*/name` for `"RPi-Sense FB"`, opens it, verifies **RGB565** (16 bpp), and `mmap`s it.
- `ShowGesture(class, confidence)` draws an 8×8 bitmap for A/B/C; class 3 (garbage) blanks the display; out-of-range draws an error "X". Color encodes confidence: **green ≥ 0.80**, **yellow ≥ 0.55**, else **red**, with brightness scaled by confidence.
- `Clear()` zeroes the matrix; the destructor clears and unmaps.

Data & control flow (stream mode)

```
LSM9DS1 ──RTIMULib──> SenseHatImu.Read ──> ring buffer (80 raw samples)
                                                  |  every 40 new samples,
when full
```

```
                                        ▼
                              trim idle prefix (motion onset, 0.15 g)
                                        ▼
                            ResampleWindow ──> 50 × 9 floats
                                        ▼
                       TfliteGestureClassifier.Predict ──> {class, confidence}
                                        ▼
                          consensus (3×) + garbage tolerance (2×) state
machine
                                        ▼
                             SenseHatDisplay.ShowGesture / Clear
```

Key tuning constants (main.cpp:28-35)

Constant	Value	Meaning
`kWindowRawSamples`	80	Raw samples kept in the ring buffer
`kWindowStride`	40	New samples between inferences
`kConsensusRequired`	3	Consecutive agreeing windows to confirm
`kPollIntervalMs`	60	IMU poll period (matches training record rate)
`kGarbageClass`	3	Index of the "no gesture" class
`kMotionThresholdG`	0.15	Dynamic-accel threshold for motion onset (local to stream loop)

Build & run

Driven by Makefile (also wired to VSCode tasks):
`make tflite` (fetch TF source) → `make train` (produce `model.tflite`) →
`make build` (cross-compile aarch64) → `make deploy` / `make run` / `make perf`
(push to the Pi over SSH). See readme.md for dev-container setup.